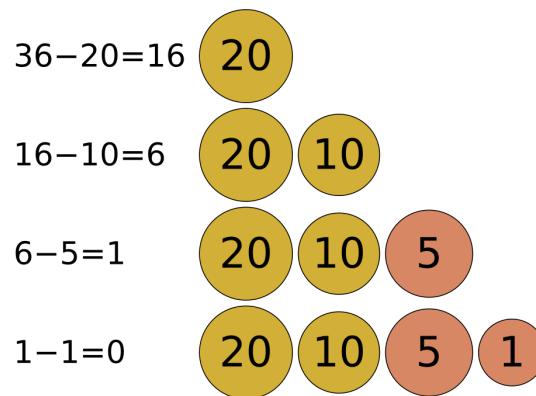


Жадный алгоритм

Жадный алгоритм ([англ. Greedy algorithm](#)) — алгоритм, заключающийся в принятии локально **оптимальных решений** на каждом этапе, допуская, что конечное решение также окажется оптимальным. Известно, что если структура задачи задается **матроидом**, тогда применение жадного алгоритма выдаст глобальный оптимум.



Жадные алгоритмы определяют минимальное количество монет, которые нужно отдать при сдаче. Показаны шаги, которые большинство людей предприняли бы для имитации жадного алгоритма для сдачи в 36 центов, используя только монеты со значениями {1, 5, 10, 20}. Монета наибольшего номинала, стоимость которой меньше, чем оставшаяся сумма сдачи, является локальным оптимумом.

Если глобальная оптимальность алгоритма имеет место практически всегда, его обычно предпочитают другим методам оптимизации, таким как [динамическое программирование](#).

Условия применимости

Общего критерия оценки применимости жадного алгоритма для решения конкретной задачи не существует, однако для задач, решаемых жадными алгоритмами, характерны две особенности: во-первых, к ним применим *Принцип жадного выбора*, а во-вторых, они обладают свойством *Оптимальности для подзадач*.

Принцип жадного выбора

Говорят, что к оптимизационной задаче применим **принцип жадного выбора**, если последовательность локально оптимальных выборов даёт глобально оптимальное решение. В типичном случае доказательство оптимальности следует такой схеме:

1. Доказывается, что жадный выбор на первом шаге не закрывает пути к оптимальному решению: для всякого решения есть другое, согласованное с жадным выбором и не хуже первого.
2. Показывается, что подзадача, возникающая после жадного выбора на первом шаге, аналогична исходной.
3. Рассуждение завершается по [индукции](#).

Оптимальность для подзадач

Говорят, что задача обладает свойством **оптимальности для подзадач для выведения результата**, если оптимальное решение задачи содержит в себе оптимальные решения для всех её подзадач. Например, в [задаче о выборе заявок](#) можно заметить, что если A — оптимальный набор заявок, содержащий заявку номер 1, то $A' = A \setminus \{1\}$ — оптимальный набор заявок для меньшего множества заявок S' , состоящего из тех заявок, для которых $s_i \leq f_1$.

Примеры

Размен монет

Задача. Монетная система некоторого государства состоит из монет достоинством $a_1 = 1 < a_2 < \dots < a_n$. Требуется выдать сумму S наименьшим возможным количеством монет.

Жадный алгоритм решения этой задачи таков. Берётся наибольшее возможное количество монет достоинства a_n : $x_n = \lfloor S/a_n \rfloor$. Таким же образом получаем, сколько нужно монет меньшего номинала, и т. д.

Для данной задачи жадный алгоритм не всегда даёт оптимальное решение, а только для некоторых, называемых *каноническими*, монетных систем, вроде используемых в США (1, 5, 10, 25 центов). Неканонические системы таким свойством не обладают. Так, например, сумму в 24 копейки монетами в 1, 5 и 7 коп. жадный алгоритм разменивает так: 7 коп. — 3 шт., 1 коп. — 3 шт., в то время как правильное решение — 7 коп. — 2 шт., 5 коп. — 2 шт. ^[1]

Выбор заявок

Формулировка № 1. Даны n заявок на проведение занятий в некоторой аудитории. В каждой заявке указаны начало и конец занятия (s_i и f_i для i -й заявки). В случае пересечения заявок можно удовлетворить лишь одну из них. Заявки с номерами i и j совместны, если интервалы $[s_i, f_i)$ и $[s_j, f_j)$ не пересекаются (то есть $f_i \leq s_j$ или $f_j \leq s_i$). Задача о выборе заявок состоит в том, чтобы набрать максимальное количество совместных друг с другом заявок.

Формулировка № 2. На [конференции](#), чтобы отвести больше времени на неформальное общение, различные секции разнесли по разным аудиториям. Учёный с чрезвычайно широкими интересами хочет посетить несколько докладов, проходящих в разных секциях. Известно начало s_i и конец f_i каждого доклада. Определить, какое максимальное количество докладов можно посетить.

Приведём жадный алгоритм, решающий данную задачу. При этом полагаем, что заявки упорядочены в порядке возрастания времени окончания. Если это не так, то можно отсортировать их за время $O(n \log n)$; заявки с одинаковым временем конца располагаем в произвольном порядке.

Функция на языке [Python](#):

```
1  def activity_selector(s, f):
2      n = len(s)  # количество занятий
3      A = [0]    # включаем первое занятие (предполагается, что
                  # занятия отсортированы по времени окончания)
4      j = 0     # индекс последней выбранной активности
5
6      for i in range(1, n):
7          if s[i] >= f[j]: # если текущее занятие начинается
                  # после окончания последнего выбранного
8              A.append(i) # добавляем его в список выбранных
9              j = i      # обновляем индекс последней выбранной
                  # активности
10
11     return A
```

На вход данному алгоритму подаются массивы начала и окончания занятий. Множество A состоит из номеров выбранных заявок, а j — номер последней заявки. Жадный алгоритм ищет заявку, начинающуюся не ранее окончания j -й, затем найденную заявку включает в A , а j присваивает её номер. Таким образом, каждый раз мы выбираем то (ещё не начавшееся) занятие, до конца которого осталось меньше всего времени.

Алгоритм работает за $O(n \log n + n)$, то есть сортировка плюс выборка. На каждом шаге выбирается наилучшее решение. Покажем, что в итоге получится оптимум.

Доказательство. Заметим, что все заявки отсортированы по неубыванию времени окончания. Заявка номер 1, очевидно, входит в оптимум (если нет, то заменим самую раннюю заявку в оптимуме на неё, от этого хуже не станет). Выкинув все заявки, противоречащие первой, получим исходную задачу с меньшим количеством заявок. Рассуждая [по индукции](#), аналогичным образом приходим к оптимальному решению.

Другие жадные алгоритмы

- [Алгоритм Хаффмана](#) (адаптивный алгоритм оптимального префиксного кодирования алфавита с минимальной избыточностью).
- [Алгоритм Крускала](#) (поиск остовного дерева минимального веса в графе).
- [Алгоритм Прима](#) (поиск остовного дерева минимального веса в связном графе).

Обобщением жадных алгоритмов является [алгоритм Радо — Эдмондса](#).

Задачи, в которых жадные алгоритмы не дают оптимального решения

Для ряда задач, относящихся к [классу NP](#), жадные алгоритмы не дают оптимального решения. К ним относятся:

- [задача коммивояжера](#);
- [задача минимальной раскраски графа](#);
- [задача разбиения графа на подграфы](#);
- [задача выделения максимальной клики](#);
- задачи, связанные с составлением [расписаний](#).

Тем не менее, в ряде задач жадные алгоритмы дают неплохие приближённые решения.

См. также

- [Жадность \(регулярные выражения\)](#)
- [Категория:Задачи, решаемые жадным алгоритмом](#)

Примечания

1. X. Cai. [Canonical Coin Systems for Change-Making Problems](http://arxiv.org/PS_cache/arxiv/pdf/0809/0809.0400v1.pdf) (http://arxiv.org/PS_cache/arxiv/pdf/0809/0809.0400v1.pdf) (англ.) // Proceedings of the Ninth International Conference on Hybrid Intelligent Systems. — 2009. — P. 499–504. — doi:10.1109/HIS.2009.103 (<https://dx.doi.org/10.1109%2FHIS.2009.103>) . — arXiv:0809.0400. Архивировано (https://web.archive.org/web/20210506124713/https://arxiv.org/PS_cache/arxiv/pdf/0809/0809.0400v1.pdf) 6 мая 2021 года.

Литература

- *Кормен, Т., Лейзерсон, Ч., Ривест, Р., Штайн, К. Глава 16. Жадные алгоритмы* (<http://www.williamspublishing.com/PDF/5-8459-0857-4/part.pdf>) // Алгоритмы: построение и анализ = Introduction to Algorithms / Под ред. И. В. Красикова. — 2-е изд. — М.: Вильямс, 2005. — 1296 с. — ISBN 5-8459-0857-4.

ССЫЛКИ

- Жадные алгоритмы (<https://web.archive.org/web/20131114234423/http://compscicenter.ru/program/lecture/5163>) Видеозапись лекции в Computer Science центре

[Сообщить об ошибке](#)